Architectures

1.Event Driven Architecture.

Event-Driven Architecture is a system design pattern where:

Services communicate by producing and consuming events.

Instead of:

Service A → calls → Service B (direct request/response)

We have:

Service A → emits event → Event Broker → Service B reacts

What is an Event?

An **event** is a record that something happened.

Examples:

- UserRegistered
- OrderCreated
- PaymentCompleted
- EmailSent

Event = Immutable fact in the past.

3️⃣ Core Components of EDA

1️⃣ Producer

Creates event.

Example:

- Order service emits OrderCreated

2️⃣ Event Broker (Message Broker)

Middleman that distributes events.

Examples:

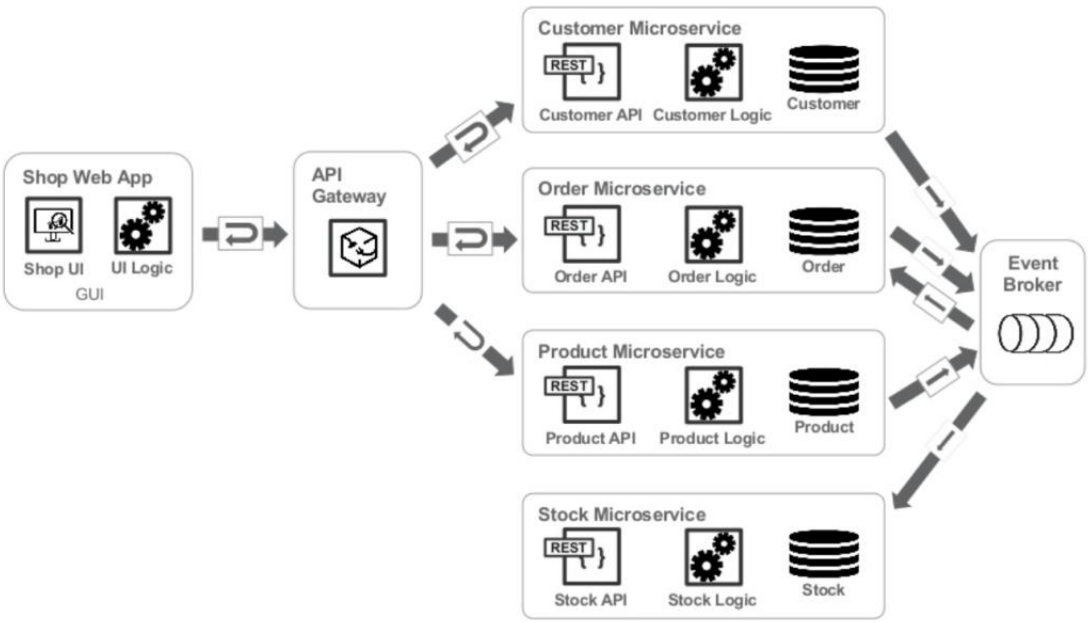
- Apache Kafka
- RabbitMQ
- Amazon SQS

3️⃣ Consumer

Listens and reacts.

Example:

- Payment service listens to OrderCreated
- Email service listens to OrderCreated



How It Works (Example)

Let's say user places an order.

Without EDA:

Order Service → calls Payment Service
→ calls Inventory Service
→ calls Email Service

Tightly coupled. If one fails → chain breaks.

With EDA:

Order Service → emits OrderCreated event

Then independently:

- Payment Service processes payment
- Inventory Service updates stock
- Email Service sends confirmation

They don’t directly depend on each other.

Benefits of Event-Driven Architecture

✅ **Loose Coupling**

Services don’t directly depend on each other.

✅ **Scalability**

Consumers scale independently.

✅ **Fault Isolation**

If email fails, payment still succeeds.

✅ **Asynchronous Processing**

Better performance for heavy workloads.

Challenges (This is Important)

EDA is powerful but complex.

❌ **Eventual Consistency**

System may not be immediately consistent.

❌ **Debugging Difficulty**

Hard to trace event chains.

❌ **Duplicate Events**

Need idempotency.

❌ **Ordering Issues**

Events may arrive out of order.

Distributed systems are hard.

When Should You Use EDA?

Use when:

- Microservices architecture
- High traffic system
- Need async workflows
- Complex business flows
- Real-time data processing

Avoid when:

- Simple CRUD app
- Small team
- Low traffic system

Event-Driven Architecture is a design pattern where services communicate by producing and consuming events instead of making direct synchronous calls. It improves scalability, fault isolation, and loose coupling. Events are sent to a message broker like Kafka or RabbitMQ, and multiple services can react independently. However, it introduces complexity like eventual consistency and debugging challenges, so it’s best suited for large distributed systems.

Layered Architecture

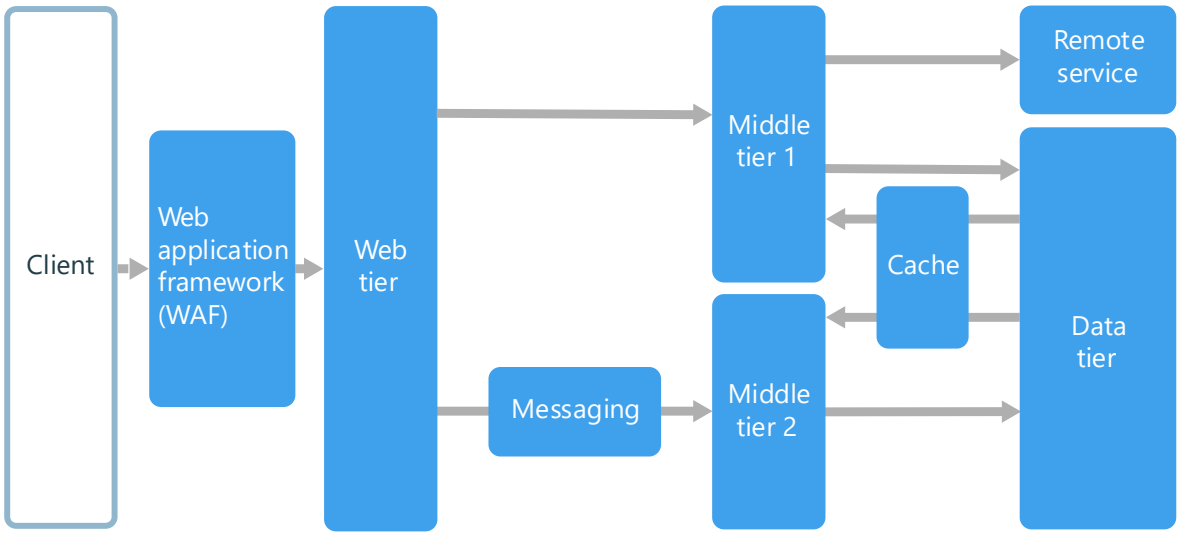
Layered Architecture (also called N-tier architecture) organizes application code into logical layers where:

Each layer has a specific responsibility and depends only on the layer below it.

2 Basic Structure (4-Layer Model)

Client
↓
Controller (Presentation Layer)
↓
Service (Business Logic Layer)
↓
Repository (Data Access Layer)
↓
Database

Each layer has one job.



🟢 1. Presentation Layer (Controller Layer)

Responsibility:

- **Handle HTTP requests**
- **Validate input**
- **Return HTTP responses**
- **No business logic**

Example (Express):

```
app.post("/users", async (req, res) => {
  const user = await userService.createUser(req.body);
  res.status(201).json(user);
});
```

Controller should:

- **Not contain database queries**
- **Not contain business rules**

It just coordinates.

2. Business Logic Layer (Service Layer)

This is the brain.

Responsibilities:

- **Core application rules**
- **Business validations**
- **Complex workflows**
- **Coordination between repositories**

Example:

```
async function createUser(data) {
  if (!data.email.includes("@")) {
    throw new Error("Invalid email");
  }
```

```
  const existing = await userRepository.findByEmail(data.email);
  if (existing) {
    throw new Error("User already exists");
  }

  return await userRepository.save(data);
}
```

Service layer should:

- Service layer should:
- Not know about HTTP
- Not return response objects
- Not handle status codes

It should only contain domain logic.

3. Data Access Layer (Repository Layer)

Responsibility:

- Communicate with database
- Execute queries
- Abstract DB implementation

Example:

```
async function findByEmail(email) {
  return await UserModel.findOne({ email });
}
```

If you switch MongoDB → PostgreSQL:

Only repository changes.
Service layer remains same.

This is powerful.

4. Database Layer

- **MongoDB**
- **PostgreSQL**
- **MySQL**
- **etc.**

It should never be accessed directly by controller.

Why Layered Architecture Is Important

✅ **1. Maintainability**

Clear structure.

✅ **2. Testability**

You can unit test service layer without Express.

✅ **3. Scalability**

Business logic is centralized.

✅ **4. Flexibility**

Change DB without rewriting app.

Dependency Rule (Very Important)

Dependencies flow downward only.

Controller → Service → Repository → DB

Never:

- **Repository calling service**
- **Service calling controller**

- **Controller accessing DB directly**

This avoids spaghetti code.

Real-World Example (Order Creation Flow)

Step-by-step flow:

Client → POST /orders

1. **Controller receives request**
2. **Controller calls orderService.createOrder()**
3. **Service validates business rules**
4. **Service calls orderRepository.save()**
5. **Repository writes to DB**
6. **Response returned**

Each layer does one job.

Folder Structure Example (Node.js)

```
src/
 ├── controllers/
 │    └── user.controller.js
 ├── services/
 │    └── user.service.js
 ├── repositories/
 │    └── user.repository.js
 ├── models/
 │    └── user.model.js
```

Advantages vs Disadvantages

Advantages:

- **Simple**

- **Industry standard**
- **Easy onboarding**
- **Great for monolith**

Disadvantages:

- **Can become tightly coupled over time**
- **Hard to scale to microservices if not modular**
- **If poorly implemented → “god service” problem**

Layered Architecture divides the application into layers like controller, service, and repository, where each layer has a specific responsibility. Controllers handle HTTP requests, services contain business logic, and repositories interact with the database. Dependencies flow downward only. This improves maintainability, testability, and separation of concerns, making it ideal for modular monolithic systems.

MVC

MVC stands for:

Model – View – Controller

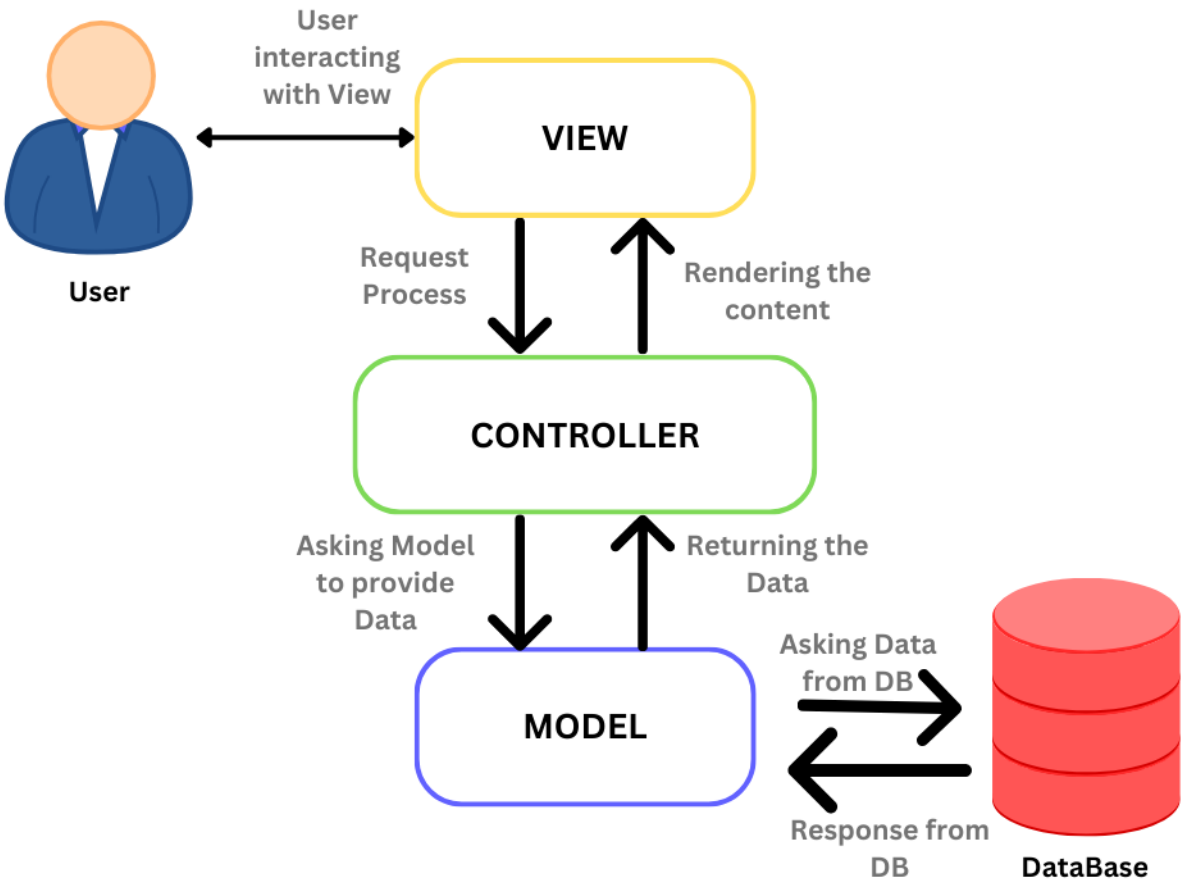
It is a design pattern that separates application into three components to enforce separation of concerns.

2️⃣ Core Idea

Instead of mixing:

- **UI**
- **Business logic**
- **Database logic**

We separate them into three roles.



1. Model

Represents:

- **Data**
- **Business rules**
- **Database interactions**

In Node.js (Mongoose example):

const User = mongoose.model("User", userSchema);

Model handles:

- **Data validation**
- **Database queries**
- **Core data logic**

2. View

Represents:

- **UI layer**
- **What user sees**

In backend-only REST API:
View = JSON response

In full-stack app:
View = HTML template / React component

Controller

Acts as middleman.

Responsibilities:

- **Receives request**
- **Calls model**
- **Returns response (view)**

Example:

```
app.get("/users/:id", async (req, res) => {
  const user = await User.findById(req.params.id);
  res.json(user);
});
```

Controller connects Model and View.

MVC vs Layered Architecture (Important)

Many people confuse them.

MVC:

Focuses on separating UI, logic, and data.

Layered:

Focuses on separating responsibilities by layers (Controller → Service → Repository).

In modern backend:

We often combine them.

Example:

Controller (MVC)
↓
Service (Layered)
↓
Repository
↓
Model

So MVC is more conceptual.
Layered is more structural.

MVC is a design pattern that separates an application into Model, View, and Controller. The Model handles data and business logic, the View handles presentation, and the Controller acts as a mediator between them. It improves separation of concerns and maintainability. In modern backend systems, MVC is often combined with layered architecture for better scalability